

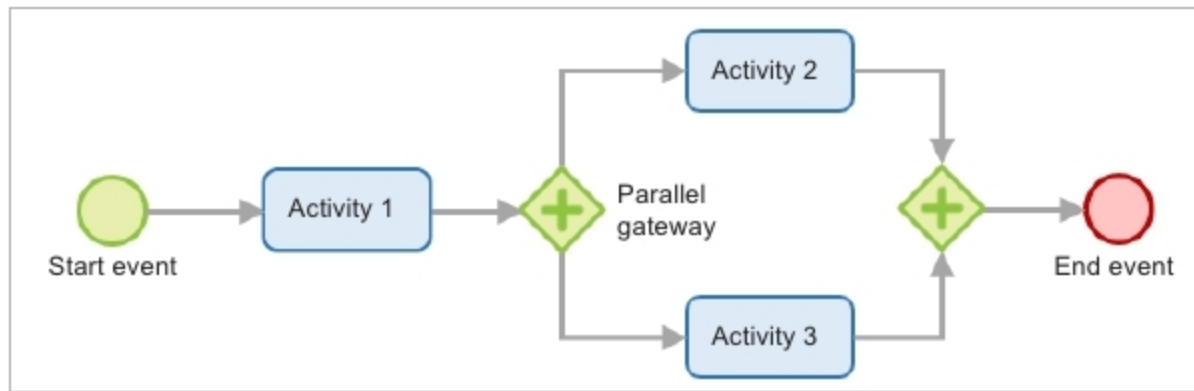


Figure 4: Basic BPMN workflow construct

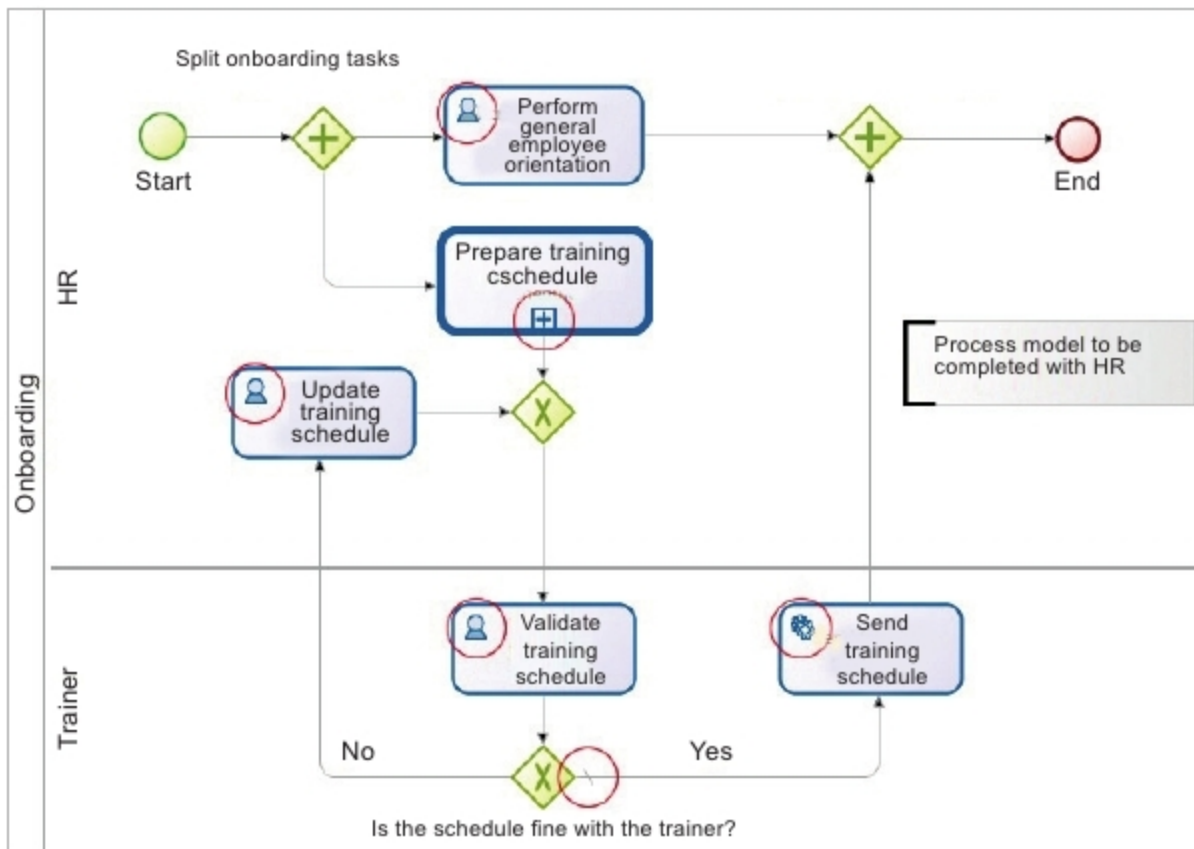


Figure 5: Generic new employee orientation and training process



Figure 6: Getting started

Signals are used to broadcast information to other processes.

Timers are symbols used to launch periodic activities to ensure that an activity happens within a specified deadline.

Errors are used to define behaviour when the system encounters an error.

Loops and multi-instance: These are used to repeat tasks such as multiple launches of the same task (multi-instance) or to repeat the same task (loop).

BPMN symbols can be defined at three levels of complexity – basic,

intermediate and advanced. While basic is predominantly visual, intermediate and advanced BPMN becomes executable. BPMN symbols serve a dual purpose. Primarily, they visually represent a process flow and secondarily, they translate to executable code that allows a visual process model to be executed as an application.

To make the visual model executable, we apply the intermediate BPMN. In an executable process, the flow model becomes an actual process application. BPMN 2.0 is not just a notation — when implemented through a modelling

tool, it provides programming instructions that a process engine uses to execute the process.

Now, let us look at our *Generic Employee Orientation and Training Process* extended with intermediate BPMN. Note that the activities have been defined and the default flow has been added.

Human activity is a step that must be done by a person, while service activity is an automated step. Call activity represents a sub-process, e.g., 'Preparing training schedule' is a call activity linked to a sub-process which is a 'child' of the original parent process. At this point in the process, a token is passed to the sub-process and when it has completed, it is passed back to the parent process. Using this notation, you can model a top-level parent process that can be quite simple. It can call a series of sub-processes that are entirely independent workflows. This means they can be modelled independently and modified as needed without necessarily changing the parent process.

In intermediate BPMN, sequence flows include conditional and default flows. When connected to inclusive or exclusive gateways, sequence flows need to be defined as conditional or default, so that the 'flow token' knows which path to follow. In conditional sequence flow, some specified condition(s) must be met so that the process can choose the next task from among two or more options. In our example, if the schedule is okay with the trainer, this condition is true and if not okay with the trainer, the condition is false.

The default sequence flow allows you to directly flow. If, for some reason, no conditions are met and the flow token always has a direction to take, default flow is marked with a \.

Sequence flow can't cross a pool boundary, so to communicate between pools (processes), use messages or signals.

Now let us start building an application with Bonita Studio.